Evidence-grounded memory in a small Go package

Cortex v1.0.1: retrieval, lifecycle, and measured limits

Brendan Playford

21 October 2026
Paper revision 2.1 | Software v1.0.1

Code examined: `b8bf2da1e06c2e2e73f8b8fbff08fa843327f551`.
Revision 2.1 edits the prose and specifies the next evaluation protocol. It adds no experimental results to revision 2.

Abstract

Cortex is a PostgreSQL-backed Go package that converts attributed observations into durable memories. This paper describes software release v1.0.1, contrasts it with the March 2026 architectural paper, and reports a local development diagnostic. The release adds optional semantic embeddings, hybrid retrieval, explicit lifecycle operations, and outcome-based utility while preserving application ownership of scheduling and authorization. On a fixed set of 30 synthetic facts across six stored runs, expected-value retrieval at five improves from 159/180 to 180/180 for topic queries and from 0/180 to 179/180 for paraphrased queries. Fresh end-to-end runs also exposed consolidation omissions: 26/30 and 30/30 intended facts were retained. The development set was used to correct the implementation, so these figures cannot estimate independent benchmark performance or downstream answer accuracy. No competing system was tested. Public benchmark evaluation remains pending.

1 Scope and relationship to the March paper

The March paper, *Applying Sleep-Dependent Memory Consolidation to Agent Architecture*, proposed a broader deployed system with scheduled dreaming, confidence decay, semantic retrieval and future learned memory policies. The present software is a reusable package. Its host application supplies observations, identity, source consent, authorization, provider credentials and the timing of maintenance. Cortex implements the state transitions and retrieval behavior.

The package exposes `Observe`, `Consolidate`, and `Recall`. Consolidation is implemented in the repository; an application invokes it after a session, after a backlog threshold, or from its own scheduler. No resident scheduler or worker is required by the library.

Release v1.0.0 deliberately retained a smaller core: attributed observations, validated insertion and reinforcement, contradiction history, and transactional acknowledgements. Release v1.0.1 restores selected lifecycle and retrieval capabilities. It does not implement the March paper's entire seven-primitive architecture or its proposed learned scorer. Paper versions and software versions are independent: this is paper revision 2.1 about software v1.0.1.

2 Evidence and transactional consolidation

An observation retains content, source references, speaker metadata and its observation timestamp. Observation idempotency is scoped to a user. A memory stores a short summary, one

of five kinds, evidence confidence, supporting observation IDs, reinforcement count and lifecycle metadata. The kinds are behavior, preference, relationship, project and routine.

The model proposes insertion, reinforcement or contradiction operations. Every fact mutation must cite exact observations from the current chunk. The implementation rejects unknown fields, duplicate JSON keys, null fields, invalid confidence values, foreign memory IDs and out-of-window evidence. An intentionally empty operations array is a valid no-op.

A PostgreSQL session advisory lock serializes consolidation for one user. The model runs without an open database transaction. Applying its validated operations, acknowledging the exact observations and completing the consolidation record happen in one transaction. A failure leaves that chunk pending; earlier committed chunks remain complete. Provider work can repeat after an interruption, but a completed chunk cannot be applied twice.

A call snapshots at most 20,000 observations and processes chunks of at most 200. At most 500 current memories enter each model context, selected by lexical overlap with the observation window, pinning and evidence recency. The result reports when this context is limited. The full serialized request remains bounded to 1 MiB, and observations are not silently truncated. Selection can miss an older related memory, so it is not a guarantee of exhaustive contradiction detection.

The consolidation record distinguishes the requested model identifier from the model returned by the provider. The default chat target is the Open Agent API alias `gpt-6-luna-medium`. A different compatible provider can be injected. Recording an alias and returned identifier improves traceability but does not guarantee immutable hosted model weights.

3 Separate confidence, validity, activation and utility

The implementation keeps four quantities separate:

- Evidence confidence estimates how well the supplied observations support a memory.
- Validity and lifecycle state determine whether the memory is eligible for current use.
- Activation represents time-dependent relevance.
- Utility estimates the observed usefulness of a retrieved memory.

A fact can remain well supported while becoming irrelevant to today's task. A preference can be superseded even when its earlier statement was reliable. Treating these changes as a single confidence number obscures their meaning.

3.1 Activation and maintenance

For an unpinned memory, activation at time t is

$$a(t) = 2^{-\frac{\max(0,t-t_e)}{H(1+\ln(\max(1,n)))}},$$

where t_e is the latest supporting observation time, H is the configured half-life, and n is the reinforcement count. Pinning sets activation to one. A zero half-life disables age decay. The default half-life is 90 days.

Activation is computed from elapsed time rather than the number of maintenance calls. Running maintenance more frequently therefore does not accelerate decay. Source evidence time is separate

from processing time: importing an old conversation must not make its evidence appear newly observed.

The explicit `Maintain` operation archives eligible memories below activation 0.05 after at least 30 days, or when their explicit expiry is reached. It processes a bounded batch and reports whether more work remains. Expiry takes precedence over pinning. These defaults have not been optimized on a public benchmark. Applications can supply another policy.

Archival is reversible and preserves evidence. It reduces the active set. The library does not run maintenance automatically or infer an expiry date for every temporary statement.

3.2 Explicit forgetting

`Forget` permanently removes the selected memory, connected contradiction versions, and memories sharing its source observations. It removes those observations, associated embeddings and feedback. Deleting a whole multi-fact observation can remove more than one fact; the returned counts make that consequence visible.

User-scoped hashes retain enough information to reject exact content or idempotency-key replays. An import barrier coordinates concurrent imports with forgetting. Index publication and retrieval recheck current memory state, including after a provider call.

This is an exact replay fence. It does not recognize every paraphrase of deleted evidence, erase already delivered provider prompts, or perform external source-permission reconciliation. The application remains responsible for consent and external deletion obligations. Every writer must be upgraded before using these operations, because an older binary does not enforce the new tombstones.

3.3 Outcome feedback

The `Feedback` operation records a user-scoped event ID, retrieval ID, memory ID, whether it was used, and an optional known success outcome. An identical event can be retried without double counting; changed contents under the same ID are rejected. Reading a memory does not reinforce it. A result left unused is not automatically a negative example.

For a used memory with a known binary outcome r, utility is updated as

$$u' = \frac{u(k+2)+r}{k+3},$$

where k is the prior outcome count. The initial utility is 0.5, corresponding to a symmetric two-observation prior. This estimator updates a stored utility value. It does not train model weights or estimate the memory's causal contribution. The legacy signal API remains available as an event log.

4 Optional semantic retrieval

An injected embedder supplies a versioned model identity and vectors through a small interface. The bundled adapter supports a standard `/embeddings` endpoint. It validates response indices, dimensions, finite values and nonzero norms. Embedding requests and chat requests may use different providers.

Vectors are stored with the model identity and memory-content hash. Indexing occurs outside database transactions. Publication checks that the memory is still current and unchanged. Failed

indexing does not roll back already committed facts; the result reports incomplete coverage and indexing can be retried.

The implementation uses exact cosine search over PostgreSQL-backed arrays. This avoids requiring another database or a vector extension. The supported active set is bounded to 10,000 memories per user; exceeding it returns an explicit capacity error. Approximate indexing is a future scale option.

Lexical token overlap and positive cosine similarity produce separate ranked candidate lists. With rank starting at one, reciprocal rank fusion gives

$$R_i = \sum_{c \in \{\text{lexical,semantic}\}} \frac{1}{60 + \text{rank}_c(i)}.$$

Absent candidates contribute zero. Each channel contributes at most four times the requested result count. The final score is

$$S_i = R_i + 10^{-5}(a_i + u_i + c_i),$$

where c_i is evidence confidence. The secondary adjustment is deliberately small.

A development diagnostic exposed an error in an earlier candidate implementation: multiplying reciprocal ranks by activation allowed an unrelated, reinforced memory to outrank the strongest semantic match. The additive adjustment corrects that failure; a regression test preserves the ordering. The next evaluation will vary retention policy separately from retrieval scoring.

Recall reports its retrieval mode, candidate count and embedding coverage. Provider failure does not silently change hybrid recall into lexical recall. The caller can request a lexical fallback explicitly. Exact supporting observations can be resolved by memory ID, but this release does not add independent semantic search over all raw observations.

5 Local evaluation

5.1 Protocol

The diagnostic uses 30 invented facts and 120 observations per run. Four chronological batches contain direct statements, attributed or quoted noise, confirmations and ten explicit corrections, and further irrelevant or adversarial input. Each stored run is queried with 30 topic-based prompts and 30 paraphrased prompts.

Two runs used each of three consolidation settings: Luna 5.6 max, Luna 5.6 medium, and Luna 6 medium. Their resulting memory sets were frozen. Retrieval comparisons use those same sets and queries. The query score checks for a case-specific expected value in the top five returned summaries. The numeric forms “40” and “forty” are treated equivalently after reviewing the outputs.

The local host was an Apple M3 Max. PostgreSQL 17 and the multi-account chat gateway ran in Docker. The semantic encoder ran locally using ONNX Runtime and `sentence-transformers/all-MiniLM-L6-v2`, revision `1110a243fdf4706b3f48f1d95db1a4f5529b4d41`. It produces 384-dimensional vectors using masked mean pooling and L2 normalization. The encoder truncates inputs at 256 wordpieces; the short diagnostic inputs fit this bound.

5.2 Results

Median query latency in the local comparison was 0.72 ms for the new lexical path and 4.99 ms for the hybrid path. These are local descriptive measurements. Cache state, scheduling and model order were not randomized, and these timings do not predict hosted-provider latency.

Retrieval system	Topic queries	Paraphrased queries
v1.0.0 lexical	159/180 (88.3%)	0/180 (0.0%)
v1.0.1 lexical	161/180 (89.4%)	0/180 (0.0%)
v1.0.1 hybrid	180/180 (100.0%)	179/180 (99.4%)

Table 1: Expected-value hits at five on frozen synthetic memory sets. The 180 queries in each column repeat 30 question templates across six stored runs; they are not 180 independent questions.

All original six consolidation runs retained the 30 intended facts, replaced the ten superseded facts, and excluded the introduced false-claim canaries. Reimporting the same observations inserted zero rows, and reconsolidation was a no-op. Two additional fresh Luna 6 runs exercised the final pipeline and automatic embedding refresh. One retained 26 of the 30 intended facts and the other retained all 30. After normalizing equivalent numeric forms, retrieval found 52/60 and 60/60 expected values respectively. The missed queries in the first run corresponded to facts omitted during consolidation. The frozen-store experiment measures retrieval after consolidation. The fresh runs also expose information lost during consolidation.

The PostgreSQL regression suite exercises migration, exact acknowledgements, interruption recovery, per-user isolation, evidence time, returned model provenance, bounded context, maintenance, feedback idempotency, forgetting/reimport, stale embedding prevention, and forgetting during retrieval. It passed with the Go race detector. Source vulnerability and secret scans also passed.

5.3 What these results do not establish

The data was created for development and was used to diagnose and correct the implementation. Its facts are short, explicit and mostly unambiguous. The near-perfect retrieval result is therefore not an unbiased estimate of real-world performance. The expected-value metric is not generated-answer accuracy, calibrated confidence, or a semantic equivalence judge.

The repetitions share templates and a narrow distribution. There is no meaningful claim of statistical significance or comparative superiority to another memory product. The evaluation corpus and raw model responses remain outside Git by the owner's instruction; the summary is not a fully reproducible public benchmark artifact.

No full LongMemEval, LoCoMo, MemoryAgentBench or Memora evaluation was run for this release. Long-running drift, mixed-language retrieval, contradictory evidence from multiple speakers, source revocation and large-scale index performance remain open.

6 Current research and next iterations

LongMemEval evaluates extraction, multi-session reasoning, time, knowledge updates and abstention [2]. MemoryAgentBench adds explicit selective-forgetting and test-time-learning competencies [3]. Memora evaluates valid-memory use alongside avoidance of obsolete information [4]. The next evaluation will measure these competencies separately.

A recent reproduction of LightMem shows that retriever choice can substantially change accuracy and that constructed memory does not consistently outperform raw-turn retrieval at matched depths [5]. Cortex should compare raw observations and summaries under equal context budgets to test whether consolidation improves answers.

FadeMem motivates selective decay and consolidation, but does not establish Cortex's default constants [6]. MemRL motivates learning utility without updating model weights [7]. Hindsight

and Graphiti provide examples of temporal evidence and multiple retrieval channels [8, 9]; their broader architectures are not requirements for this package.

7 Next evaluation and release plan

This protocol is proposed work. None of the benchmark runs or implementation changes below contributes to the results in Section 5.

7.1 Freeze the baseline and verify the research

Keep software v1.0.1 at the commit printed on the first page. Record provider model identifiers, embedding revision, database schema and configuration with every run. Use Scarlett to retrieve the cited primary papers and their official benchmark repositories. The LongMemEval, LightMem reproduction and Memora paper records were checked through Scarlett on 21 October 2026; the benchmark runs remain pending. Check the source dates and benchmark licenses before downloading or running a corpus. A failed research request supplies no evidence about the paper it was meant to retrieve.

7.2 Measure the missing-fact failure before changing extraction

Create a held-out synthetic set with explicit multi-fact statements, corrections, quoted material, irrelevant statements and ambiguous claims. Define the expected facts and acceptable abstentions before running consolidation. Keep those cases separate from the 30 development facts in this paper. Record omissions, unsupported additions, wrong-speaker attribution and incorrect supersession, then inspect which operation lost each fact.

Run the baseline and each candidate against the same observations. Compare a narrow extraction correction with an optional raw-observation retrieval path. Any raw result must retain its original source and speaker, respect the same user and deletion boundaries, and remain distinguishable from a consolidated memory. Adopt a candidate only if it fixes the recorded omission cases without adding unsupported facts or breaking existing invariants.

7.3 Run independent retrieval and answer comparisons

Start with LongMemEval-S using its official evaluation procedure. Freeze the reader, judge configuration, corpus split, prompts and input-token budget before comparing raw observations, consolidated hybrid retrieval and the candidate fallback. Report retrieval coverage separately from answer accuracy, temporal updates and abstention. Use repeated runs to expose model variation; disclose run count and any deviations from the benchmark procedure.

Run integration tests in disposable local PostgreSQL and API-gateway containers. Use synthetic identities with the available provider credentials and Luna 6 medium, recording the requested and returned model names for each run. Exercise credential failover, timeouts and retries. Report provider errors separately from answer quality, together with latency, token usage and available cost measurements. Credentials, local sessions, corpus contents, prompts, raw responses and database exports stay outside Git.

7.4 Test lifecycle policy and feedback separately

Compare decay enabled and disabled while holding retrieval and inputs fixed. Include an old durable fact, an expired temporary fact, a pinned memory, a corrected preference and a deleted

observation that is replayed. Verify that elapsed time changes activation without rewriting evidence confidence, explicit expiry takes precedence over pinning, and forgotten content stays unavailable across imports and concurrent retrieval.

Exercise identical and conflicting feedback retries. Retrieval alone must not reinforce a fact, and an unused result must not become a negative outcome. Any cross-user disclosure, unsupported write, duplicate outcome update or resurrection of deleted evidence fails the release checks. Use relevant MemoryAgentBench and Memora tasks to extend the evaluation after the first held-out comparison.

7.5 Release only the changes supported by the comparison

Keep consolidation and maintenance as explicit host calls. Preserve the optional embedder and avoid adding a resident worker, graph store or learned policy without a measured need. Benchmark the existing exact index at its documented active-set limit before considering an approximate index.

A correction confined to existing behavior can use a patch release. A new public retrieval option should use a minor release, proposed as v1.1.0. Before publication, run migrations, PostgreSQL integration and race tests, vulnerability and secret scans, and clean-install smoke tests. Record remaining failures alongside the aggregate results. Publish a new paper revision with its software tag, exact commit, protocol and limitations. Preserve this revision and the unchanged March paper so readers can distinguish design history from tested behavior.

References

[1] B. Playford. *Applying Sleep-Dependent Memory Consolidation to Agent Architecture*. March 2026. Archived original paper, version 1.

[2] D. Wu et al. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. ICLR 2025. https://arxiv.org/abs/2410.10813.

[3] Y. Hu, Y. Wang, and J. McAuley. *Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions*. Revised June 2026. https://arxiv.org/abs/2507.05257.

[4] M. N. Uddin et al. *From Recall to Forgetting: Benchmarking Long-Term Memory for Personalized Agents*. 2026. https://arxiv.org/abs/2604.20006.

[5] Y. Zhou, S. Wang, B. Koopman, and G. Zuccon. *Reproducing LightMem: When Does Memory Construction Help Over Raw-Turn Retrieval?* Revised 17 October 2026. https://arxiv.org/abs/2607.29104.

[6] L. Wei et al. *FadeMem: Biologically-Inspired Forgetting for Efficient Agent Memory*. 2026. https://arxiv.org/abs/2601.18642.

[7] S. Zhang et al. *MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory*. 2026. https://arxiv.org/abs/2601.03192.

[8] C. Latimer et al. *Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects*. 2025. https://arxiv.org/abs/2512.12818.

[9] Zep. *Graphiti: a framework for temporal context graphs*. Source and documentation accessed 21 October 2026. https://github.com/getzep/graphiti.

[10] B. Playford. *Cortex v1.0.1*. Code release. https://github.com/teslashibe/cortex/releases/tag/v1.0.1.